

Manual técnico

Lenguajes y tecnologías

- JavaScript: lógica del frontend y backend.
- HTML: estructura base en frontend/index.html.
- CSS: estilos base.
- Prisma Schema: modelado de la base de datos.
- SQLite: base de datos actual.

Paquetes principales

- Frontend:
 - react: interfaz de usuario.
 - react-dom: renderizado en navegador.
 - react-router-dom: navegación por rutas.
 - vite: entorno de desarrollo y build.
 - tailwindcss, postcss, autoprefixer: estilos.
 - eslint: calidad de código.

Backend:

- express: API REST.
- cors: permitir conexión frontend-backend.
- dotenv: variables de entorno.
- jsonwebtoken: autenticación con token.
- prisma, @prisma/client: acceso y manejo de base de datos.
- pg: driver de PostgreSQL, instalado pero no parece ser el usado ahora.

Arquitectura

- Arquitectura por capas.
- No es MVC puro ni microservicios.
- Se divide en:
 - Frontend: presentación.
 - Backend: lógica de negocio y API.
 - Base de datos: persistencia con Prisma.

Carpetas principales

- frontend/: aplicación web.
- backend/: servidor y API.
- backend/prisma/: esquema y datos iniciales.
- node_modules/: dependencias instaladas.
- package.json: scripts y dependencias generales del proyecto.

Responsabilidad de cada carpeta

- frontend/src/pages: pantallas principales como login, admin, cajero y mesero.
- frontend/src/utills: utilidades del cliente, como sesión e íconos.
- frontend/public: recursos estáticos.
- backend/src/routes: endpoints de usuarios, menú, mesas, pedidos y reportes.
- backend/src/utills: lógica de apoyo, como manejo de turnos.
- backend/prisma/schema.prisma: modelos de datos.
- backend/src/index.js: arranque del servidor y registro de rutas.

Qué hace cada módulo o paquete

- userRoutes: gestión de usuarios y roles.
- menuRoutes: categorías y productos.
- tableRoutes: manejo de mesas y estados.
- orderRoutes: creación de pedidos, cobros, domicilios y anulaciones.
- reportRoutes: ventas, caja, turnos, cierres y reportes.
- authMiddleware: validación de token y permisos.
- pinSecurity: login por PIN.
- shiftHelper: apoyo para apertura/cierre de turnos.

¿Cómo se comunican entre sí?

1. El usuario interactúa con el frontend.
2. El frontend hace peticiones HTTP al backend.
3. El backend procesa reglas de negocio.
4. El backend consulta o actualiza la base de datos con Prisma.
5. El backend responde al frontend con JSON.
6. El frontend muestra la información según el rol del usuario.

En conclusión

Este proyecto consiste en un sistema de punto de venta (POS) para un restaurante. Su objetivo es facilitar la gestión de pedidos, productos y ventas.

El sistema está construido como una sola aplicación donde todas sus partes funcionan juntas. Sin embargo, está organizado en diferentes secciones según su función.

Por un lado, se encuentra la interfaz de usuario desarrollada con React, que permite a los usuarios interactuar con el sistema. Por otro lado, la lógica del sistema está implementada con Express, que se encarga de procesar la información y manejar las operaciones.

Finalmente, la información se almacena en una base de datos gestionada mediante Prisma y SQLite.

2. Explicación de APIs o endpoints

Endpoint	Entrada	Proceso	Salida	Rol autorizado
POST /api/auth/login	{ pin }	Valida PIN y genera token JWT	{ message, token, user }	Todos
GET /api/users	Token	Consulta lista de usuarios	Arreglo de usuarios	ADMIN
POST /api/users	{ nombre, pin_acceso, rol }	Crea un nuevo usuario	{ message, user }	ADMIN
PUT /api/users/:id	id + campos a editar	Actualiza datos del usuario	Usuario actualizado	ADMIN
DELETE /api/users/:id	id	Elimina usuario	{ message }	ADMIN
GET /api/menu/categories	Ninguna	Consulta categorías del menú	Arreglo de categorías	Todos en la app
POST /api/menu/categories	{ nombre }	Crea una categoría	Categoría creada	ADMIN
PUT /api/menu/categories/:id	id, { nombre }	Actualiza categoría	Categoría actualizada	ADMIN
DELETE /api/menu/categories/:id	id	Elimina categoría	{ message } o error	ADMIN
GET /api/menu/products	Ninguna	Consulta productos disponibles/no archivados	Arreglo de productos	Todos en la app
POST /api/menu/products	{ nombre, precio, id_categoria, imagen }	Crea producto	Producto creado	ADMIN
PUT /api/menu/products/:id	id + campos	Actualiza producto	Producto actualizado	ADMIN
DELETE /api/menu/products/:id	id	Elimina o archiva producto	{ message }	ADMIN
GET /api/tables	Ninguna	Consulta mesas del restaurante	Arreglo de mesas	Todos en la app
POST /api/tables	{ numero_mesa, capacidad }	Crea mesa	Mesa creada	ADMIN
PUT /api/tables/:id	id, { numero_mesa, capacidad }	Actualiza mesa	Mesa actualizada	ADMIN
PATCH /api/tables/:id/status	id, { estado }	Cambia estado de la mesa	Mesa actualizada	ADMIN, CAJERO, MESERO
DELETE /api/tables/:id	id	Elimina mesa si está libre	{ message }	ADMIN
POST /api/orders	{ tipo_pedido, id_mesa, datos_cliente, items }	Crea orden nueva o agrega productos a una existente	{ message, order }	MESERO, CAJERO
GET /api/orders/mesa/:id	id de mesa	Busca cuenta abierta de la mesa	Orden abierta	MESERO, CAJERO
GET /api/orders/delivery	Ninguna	Consulta domicilios activos	Arreglo de órdenes delivery	CAJERO
PATCH /api/orders/:id/status	id, { estado }	Cambia estado de la orden	Orden actualizada	Usuarios autenticados
POST /api/orders/:id/pay	id, { metodo_pago, monto_pagado, referencia }	Registra pago, cierra orden y libera mesa	{ message }	CAJERO, ADMIN
POST /api/orders/:id/void-item/:itemId	id, itemId, { motivo, cantidad? }	Anula un producto de la orden	{ message }	CAJERO, ADMIN
POST /api/orders/:id/void	id, { motivo }	Anula la orden completa	{ message }	CAJERO, ADMIN
GET /api/reports/sales	Ninguna	Genera reporte diario de ventas	{ resumenDiario, topProducts, tickets, pagos, voidRecords }	ADMIN, CAJERO
GET /api/reports/current-shift	Ninguna	Consulta resumen del turno actual	Datos del turno actual	CAJERO, ADMIN
GET /api/reports/check-open-shift	Ninguna	Verifica si existe turno abierto	{ hasOpenShift, shiftId, fondo_inicial }	CAJERO, ADMIN
POST /api/reports/open-shift	{ fondo_inicial }	Abre un nuevo turno de caja	{ message, shift }	CAJERO, ADMIN
POST /api/reports/cash-movement	{ tipo, monto, motivo }	Registra ingreso o egreso manual ↓	{ message, movement }	CAJERO, ADMIN
POST /api/reports/cierre	{ efectivo_contado }	Cierra turno y calcula descuadre	{ message, descuadre }	CAJERO, ADMIN

3. Base de datos (modelo entidad-relación)

Entidad	Función	Atributos clave	Relación
User	Guarda los usuarios del sistema	id, nombre, pin_acceso, rol	Se relaciona con Order, Payment, VoidRecord y Shift
Table	Representa las mesas del restaurante	id, numero_mesa, capacidad, estado	Se relaciona con Order
Category	Agrupar los productos del menú	id, nombre	Se relaciona con Product
Product	Guarda los platos o productos	id, nombre, precio, id_categoria, disponible	Pertenece a Category y se relaciona con OrderItem
Order	Registra los pedidos realizados	id, tipo_pedido, id_mesa, id_mesero, estado, total	Se relaciona con User, Table, OrderItem y Payment
OrderItem	Guarda el detalle de cada producto pedido	id_orden, id_producto, cantidad, precio_unitario	Pertenece a Order y Product
Payment	Registra los pagos de las órdenes	id_orden, id_cajero, metodo_pago, monto_pagado	Se relaciona con Order, User y Shift
VoidRecord	Guarda el historial de anulaciones	id_cajero, id_turno, tipo, motivo	Se relaciona con User y Shift
Shift	Representa los turnos de caja	id_usuario, fecha_apertura, fecha_cierre, fondo_inicial	Se relaciona con User, Payment, VoidRecord y CashMovement
CashMovement	Registra ingresos y egresos manuales de caja	id_turno, tipo, monto, motivo	Pertenece a Shift

Base de datos

Este proyecto utiliza una base de datos relacional mediante SQLite, gestionada a través de Prisma.

La información se organiza en tablas que se relacionan entre sí, lo que permite manejar de forma estructurada los datos del sistema, como usuarios, productos, mesas, pedidos y pagos.

Entidades principales

Usuario: Almacena la información de las personas que utilizan el sistema.

Atributos: id_usuario, nombre, pin_acceso, rol.

Categoría: Permite organizar los productos del menú.

Atributos: id_categoria, nombre.

Producto: Contiene la información de los productos disponibles.

Atributos: id_producto, nombre, precio, imagen, disponibilidad, id_categoria.

Mesa: Representa las mesas del restaurante.

Atributos: id_mesa, numero_mesa, capacidad, estado.

Orden-Pedido: Registra los pedidos realizados por los clientes.

Atributos: id_orden, tipo_pedido, estado, fecha, id_mesa, id_usuario.

Detalle_Orden: Relaciona los productos incluidos en cada pedido.

Atributos: id_detalle, id_orden, id_producto, cantidad, precio_unitario, subtotal.

Pago: Registra la información de los pagos realizados.

Atributos: id_pago, id_orden, metodo_pago, monto_pagado, referencia.

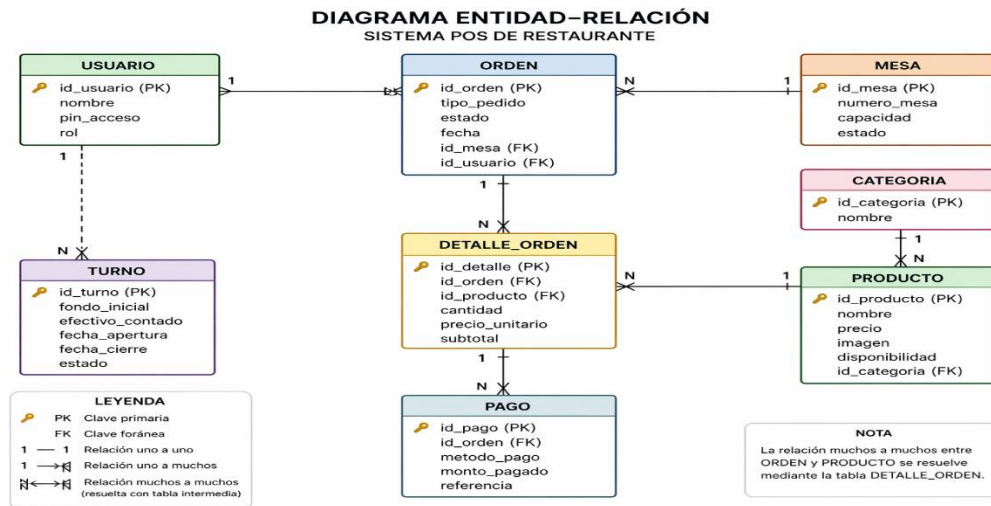
Turno: Gestiona la apertura y cierre de caja.

Atributos: id_turno, fondo_inicial, efectivo_contado, fecha_apertura, fecha_cierre, estado.

Relaciones entre entidades

- Un **usuario** puede registrar múltiples órdenes.
→ Relación: uno a muchos.
- Una **categoría** puede contener múltiples productos.
→ Relación: uno a muchos.
- Una **mesa** puede tener múltiples órdenes a lo largo del tiempo.
→ Relación: uno a muchos.
- Una **orden** puede incluir varios productos, y un producto puede pertenecer a varias órdenes.
→ Relación: muchos a muchos, resuelta mediante la tabla **Detalle_Orden**.
- Una **orden** puede tener uno o varios pagos, dependiendo de la implementación.
→ Relación: uno a uno o uno a muchos.

4. Diagrama



5. Guía de mantenimiento y escalabilidad

- **Corregir errores:** identificar el problema, revisarlo en el sistema, corregirlo y probar que no afecte otras funciones.
- **Actualizar dependencias:** mantener librerías al día, pero probar primero los cambios antes de usarlos en producción.
- **Probar cambios:** verificar funciones principales como login, usuarios, pedidos, pagos y reportes.
- **Escalabilidad:** optimizar la base de datos y el servidor para soportar más usuarios al mismo tiempo.
- **Posibles mejoras:** usar caché, dividir el sistema en servicios y agregar monitoreo y respaldos.

6. Control de versiones

Actualmente, el proyecto no tiene un repositorio Git configurado, por lo que no se lleva un control formal de los cambios en el código. Aun así, sería recomendable usar Git para guardar versiones del proyecto, organizar mejor las modificaciones y evitar perder avances. Como forma de trabajo, se podrían usar ramas como `main` para la versión estable, `develop` para cambios en proceso y `feature` para nuevas funciones o correcciones.

Flujo de trabajo

1. Se crea una rama feature para trabajar un cambio sin afectar la versión principal.
2. Cuando el cambio está listo y probado, se une a develop.
3. En develop se revisan e integran varias mejoras.
4. Cuando todo funciona bien, los cambios pasan a main, que es la versión final o estable.